ÉCOLE NATIONALE D'INGÉNIEURS DE TUNIS

Département Génie Industriel

Rapport de projet de probabilités

Elaboré par :

Gargouri Ismail

Abdennadher Amine

Bejaoui Saad eddine

Encadré par :

M. MAKHLOUF Azmi

1ère Année MIndS

Année universitaire : 2024/2025

Table des matières

Table des figures **4**

Introduction Générale **2**

1 Résolution de soduku **4**
1.1 Introduction 4
1.2 Méthodologie 4
1.2.1 Algorithme de Metropolis-Hastings appliqué au Sudoku . . . 4
1.2.2 Méthode de Monte Carlo 5
1.3 Analyse des résultats 5
1.4 Conclusion 6

2 Extension : Exploration d'un labyrinthe par Metropolis-Hastings **8**
2.1 Introduction 8
2.2 Modélisation et méthodologie 8
2.3 Analyse des données 11
2.4 Conclusion 12

3 Cristallographie computationnelle : Analyse Thermodynamique d'un cristal de Lennard-Jones à l'aide de Metropolis-Hastings **13**
Introduction 13
3.1 Fondements théoriques 14
3.1.1 Potentiel de Lennard Jones : modèle d'intéractions atomiques 14
3.1.2 Unités réduites : simplification et généralisation des simulations 15
3.1.3 Recuit Simulé pour la Relaxation Énergétique 16
3.1.4 Fonction de Distribution Radiale 17
3.2 Implémentation numérique 18
3.2.1 Langage et environnement utilisés 18
3.2.2 Algorithme général de la simulation 18
3.2.3 Organisation du code 19
3.2.4 Programme principal 19

3.3 Résultats et analyses 20
3.3.1 Structure la plus stable d'un cristal avec défauts 20
3.3.2 Transition de phase solide-liquide 23
3.4 Conclusion 25

4 Prédiction bayésienne du prix d'une cryptomonnaie par échantillonnage MCMC : Application de l'algorithme de Metropolis-Hastings 26
4.1 Introduction 26
4.2 Fondements théoriques 27
4.2.1 Loi normale pour les rendements 27
4.2.2 Vraisemblance et fonction d'énergie 27
4.2.3 Algorithme de Metropolis-Hastings 28
4.3 Mise en œuvre algorithmique 28
4.3.1 Génération des données 28
4.3.2 Paramètres et initialisation 29
4.3.3 Implémentation du Metropolis-Hastings 29
4.3.4 Stockage des échantillons et prédiction 30
4.3.5 Visualisations 30
4.4 Analyse des résultats 31
4.4.1 Convergence de la chaîne 31
4.4.2 Résultats statistiques 32
4.5 Comparaison avec un modèle naïf 33
4.5.1 Comparaison des résultats 33
4.6 Discussion et limites 34
4.6.1 Sensibilité aux hyperparamètres 34
4.6.2 Simplifications du modèle 34
4.6.3 Axes d'amélioration 34
4.7 Conclusion 35

Conclusion Générale 36

Bibliographie 38

Table des figures

2.1 exemple de résolution d'une labyrinthe 11
2.2 l'histogramme : "Distribution des distances à la sortie" 12

3.1 Configuration initiale avec défaut cristallin. 21
3.2 configuration initiale avec défaut cristallin. 21
3.3 Structure la plus stable obtenue après relaxation. 22
3.4 Distribution radiale $g(r)$ de la configuration optimisée. 22
3.5 Évolution de la structure cristalline et de la fonction $g(r)$ au cours de la simulation. 24

4.1 Évolution des paramètres μ et σ au cours des itérations 31
4.2 Évolution de l'énergie au cours des itérations 32
4.3 Comparaison entre la distribution du prix futur de Bitcoin au jour suivant pour le modele naïf et l'approche bayésienne. 33

Remerciement

Nous tenons à exprimer notre profonde gratitude envers vous pour avoir été notre encadreur monsieur MAKHLOUF Azmi tout au long de notre projet de fin d'études. Votre guidance, vos conseils éclairés et votre soutien constant ont joué un rôle déterminant dans la réussite de ce projet.

Nous voudrions également étendre nos remerciements à toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce PFA. Chacun de vous a apporté une expertise précieuse et a contribué à enrichir ce travail de manière significative.Merci encore pour votre engagement et votre expertise.

Introduction Générale

La complexité croissante des systèmes réels dans des domaines aussi variés que les jeux logiques, l'exploration spatiale, la physique des matériaux ou encore la finance, nécessite des outils mathématiques capables de modéliser l'incertitude, de gérer de grands espaces d'états, et de fournir des solutions approchées là où les approches déterministes échouent. C'est dans cette perspective que s'inscrit ce projet, qui met à l'honneur une méthode probabiliste puissante et polyvalente : l'algorithme de Metropolis-Hastings, appartenant à la famille des méthodes de Monte Carlo par chaînes de Markov (MCMC). L'objectif de ce travail est d'explorer l'efficacité et l'adaptabilité de cette méthode dans des contextes variés, en mettant l'accent sur sa capacité à échantillonner efficacement des distributions complexes. Chaque chapitre du rapport illustre une application concrète, où la modélisation probabiliste est indispensable à la compréhension ou à la résolution du problème.

Dans un premier temps, l'algorithme est appliqué à la résolution du Sudoku, démontrant sa capacité à explorer un espace de configurations sous contraintes. Cette approche combine ingénieusement l'échantillonnage probabiliste avec une vérification des règles logiques du jeu.

La deuxième partie s'intéresse à l'exploration d'un labyrinthe, problématique emblématique de la robotique et de l'intelligence artificielle. Metropolis-Hastings y est utilisé comme stratégie de navigation stochastique, simulant des parcours plausibles vers une sortie, dans un environnement incertain.

La troisième partie aborde un sujet de physique computationnelle : la simulation thermodynamique d'un cristal de Lennard-Jones. Dans ce contexte, l'algorithme permet de modéliser l'évolution énergétique d'un système atomique, d'étudier la structure la plus stable d'un cristal et d'observer des phénomènes de transition de phase.

Enfin, le dernier chapitre propose une application en finance bayésienne, à travers la prédiction du prix d'une cryptomonnaie (Bitcoin). Grâce à une modélisation des rendements comme variables aléatoires, Metropolis-Hastings permet d'estimer la distribution a posteriori des paramètres et d'obtenir des prévisions probabilistes du prix futur, avec incertitude intégrée.

Ce rapport montre ainsi que l'algorithme de Metropolis-Hastings ne se limite pas

à un domaine particulier, mais s'impose comme un outil central de la modélisation probabiliste, offrant des solutions pertinentes dans des univers aussi différents que le jeu, la physique, l'intelligence artificielle et la finance. Il reflète également l'intérêt croissant pour les méthodes bayésiennes dans la résolution de problèmes réels où l'incertitude joue un rôle fondamental.

Chapitre 1

Résolution de soduku

1.1 Introduction

Ce projet a pour objectif de résoudre des grilles de Sudoku en utilisant l'algorithme de Metropolis-Hastings, une méthode de Monte Carlo par chaînes de Markov (MCMC). Cette approche permet d'explorer efficacement un espace de solutions très vaste et contraint.

L'idée est de générer progressivement des grilles candidates, en acceptant ou rejetant des modifications selon un critère probabiliste. Cela permet à l'algorithme de converger vers une solution valide du Sudoku.

Nous utiliserons également des simulations Monte Carlo pour estimer la durée moyenne de résolution et construire des intervalles de confiance, afin d'évaluer la performance de l'algorithme de manière statistique.

1.2 Méthodologie

1.2.1 Algorithme de Metropolis-Hastings appliqué au Sudoku

La grille de Sudoku est représentée comme une matrice 9x9. Les cases déjà remplies sont fixes, et ne doivent jamais être modifiées. Pour initialiser la grille, on complète chaque bloc 3x3 avec les chiffres manquants de manière aléatoire, tout en évitant les doublons à l'intérieur du bloc.

À chaque itération, on choisit un bloc 3x3 au hasard et on échange deux cases variables. Cela constitue une nouvelle proposition. La fonction d'énergie mesure combien de contraintes du Sudoku (lignes, colonnes) sont violées.

Le critère d'acceptation est basé sur une loi de probabilité liée à cette énergie. Une grille avec moins de violations est plus probable d'être acceptée. Si la grille

est meilleure, elle est toujours acceptée. Sinon, elle est acceptée avec une certaine probabilité.

1.2.2 Méthode de Monte Carlo

Pour évaluer la performance de l'algorithme, on mesure le temps nécessaire pour résoudre une grille, sur un grand nombre de simulations indépendantes. On obtient ainsi une moyenne empirique et on calcule un intervalle de confiance à 95

L'intervalle est basé sur la moyenne des temps et leur écart-type. Cela permet de quantifier l'incertitude de l'estimation et de comparer la difficulté de différentes grilles.

1.3 Analyse des résultats

Nous avons testé l'algorithme sur plusieurs grilles de Sudoku avec différents niveaux de difficulté. Pour chaque grille, nous avons lancé 10 simulations et mesuré le temps nécessaire pour atteindre une solution valide ou le nombre d'itérations nécessaires.

Les résultats montrent que l'algorithme réussit à résoudre les grilles simples rapidement, avec une énergie qui converge rapidement vers zéro. Pour les grilles plus difficiles, le temps moyen augmente et l'écart-type est plus élevé, ce qui reflète une variabilité plus grande.

Table 1.1 – Nombre d'itérations par exécution (affichage horizontal)

Exécution	1	2	3	4	5	6	7	8	9	10
Itérations	1300	278	405	145	802	375	2307	274	195	538

1. Moyenne

La moyenne empirique est donnée par :

$$\bar{x} = \frac{1}{10}\sum_{i=1}^{10} x_i = \frac{6619}{10} = \boxed{661{,}9}$$

2. Variance

La variance empirique est :

$$s^2 = \frac{1}{10}\sum_{i=1}^{10}(x_i - \bar{x})^2 = \boxed{406\,999{,}89}$$

3. Écart-type

$$s = \sqrt{406\,999{,}89} = \boxed{637{,}98}$$

4. Intervalle de confiance à 95%

Avec $n = 10$, on utilise la formule :

$$IC_{95\%} = \left[\bar{x} - 1{,}96 \cdot \frac{s}{\sqrt{n}},\ \bar{x} + 1{,}96 \cdot \frac{s}{\sqrt{n}}\right]$$

$$= \left[661{,}9 - 1{,}96 \cdot \frac{637{,}98}{\sqrt{10}},\ 661{,}9 + 1{,}96 \cdot \frac{637{,}98}{\sqrt{10}}\right]$$

$$\approx [266{,}5\ ;\ 1057{,}3]$$

5. Interprétation des résultats

L'algorithme met en moyenne environ 662 itérations pour résoudre la grille. Cependant, l'écart-type élevé (presque 638) montre une forte dispersion des résultats, ce qui est confirmé par l'intervalle de confiance large : [266.5 ; 1057.3].

Cela signifie que les performances de l'algorithme sont très sensibles à l'aléatoire : selon les valeurs initiales et les propositions faites, l'algorithme peut converger très rapidement (ex. 145 itérations) ou prendre beaucoup plus de temps (ex. 2307 itérations).

Cette variabilité justifie pleinement l'usage de simulations Monte Carlo pour estimer la moyenne et encadrer statistiquement les résultats.

1.4 Conclusion

L'algorithme de Metropolis-Hastings s'adapte bien à la résolution probabiliste du Sudoku. Il permet d'explorer efficacement l'espace des solutions, même s'il ne garantit pas toujours une solution optimale dans un temps limité.

Grâce à la méthode de Monte Carlo, nous avons pu estimer la performance de manière rigoureuse, en tenant compte de l'aléa introduit par les choix initiaux et les propositions.

Ce travail pourrait être étendu en améliorant la stratégie de proposition, en adaptant dynamiquement la température, ou en appliquant la méthode à d'autres problèmes de satisfaction de contraintes.

Chapitre 2

Extension : Exploration d'un labyrinthe par Metropolis-Hastings

2.1 Introduction

La résolution de labyrinthes représente un problème classique en algorithmique et en intelligence artificielle. Dans ce projet, nous abordons le problème à l'aide d'une méthode probabiliste : l'algorithme de Metropolis-Hastings. Cet algorithme est traditionnellement utilisé pour l'échantillonnage selon des distributions complexes, mais ici, il est détourné pour guider un agent à travers un labyrinthe en fonction d'une densité cible favorisant les positions proches de la sortie. L'objectif est de comparer cette approche stochastique à des méthodes classiques de parcours, en étudiant notamment la distribution des positions visitées.

2.2 Modélisation et méthodologie

Le labyrinthe est généré de manière procédurale par une méthode de type "depth-first search" (DFS) afin de garantir un graphe connexe avec une unique solution. Il est modélisé sous forme de matrice binaire, où les cellules à 0 représentent les chemins accessibles et les cellules à 1 les murs. Cela est assuré par la fonction "generatemaze()" comme montré ci-dessous :

```
function maze = generateMaze(n)
    maze = ones(n); % murs partout
    maze(2:2:end-1, 2:2:end-1) = 0; % cellules accessibles

    visited = zeros(size(maze));
    stack = [2 2];
    visited(2,2) = 1;

    moves = [0 2; 0 -2; 2 0; -2 0];

    while ~isempty(stack)
        current = stack(end,:);
        neighbors = [];

        for i = 1:4
            new = current + moves(i,:);
            if all(new > 1 & new < n) && ~visited(new(1),
               new(2))
                neighbors = [neighbors; new];
            end
        end

        if ~isempty(neighbors)
            next = neighbors(randi(size(neighbors,1)), :);
            wall = (current + next) / 2;
            maze(wall(1), wall(2)) = 0;
            visited(next(1), next(2)) = 1;
            stack = [stack; next];
        else
            stack(end,:) = []; % backtrack
        end
    end
end
```

Listing 2.1 – Script `fonction generatemaze()`

L'algorithme de Metropolis-Hastings est utilisé pour simuler un déplacement dans le labyrinthe :

À chaque étape, une position voisine est proposée aléatoirement.

Si la case est accessible, la transition est acceptée avec une probabilité liée à la densité cible.

La densité cible est une fonction décroissante de la distance à la sortie, favorisant donc l'exploration des zones proches de celle-ci.

Les positions successives de l'agent sont enregistrées, ce qui permet ensuite de visualiser sa trajectoire ainsi que la fréquence des visites dans l'espace.

```
function metropolis_maze_solver()
    clear; clc; close all;

    n = 21; % Doit  tre  impair pour bon fonctionnement de la maze
    maze = generateMaze(n);

    % Coordonn es d part / arriv e
    start = [2, 2];
    goal = [n-1, n-1];

    % Fonction cible : plus on est proche de la sortie, plus la proba est haute
    target_pdf = @(pos) exp(-0.3 * norm(pos - goal));

    % Initialisation Metropolis-Hastings
    current_pos = start;
    samples = [];
    N = 8000;

    % D placements (haut, bas, gauche, droite)
    moves = [0 1; 0 -1; 1 0; -1 0];

    for i = 1:N
        move = moves(randi(4), :);
        proposal = current_pos + move;

        if all(proposal >= 1) && all(proposal <= n) && maze(proposal(1), proposal(2)) == 0
            p_current = target_pdf(current_pos);
            p_proposal = target_pdf(proposal);
            alpha = min(1, p_proposal / p_current);

            if rand < alpha
                current_pos = proposal;
            end
        end
        samples = [samples; current_pos];
    end
```

Listing 2.2 – Script `fonction metropolismazesolver()`

2.3 Analyse des données

La Figure 1 montre le labyrinthe avec la trajectoire de l'agent (en rouge), le point de départ (vert) et la sortie (bleu). On observe que, malgré la nature aléatoire de l'algorithme, l'agent suit une trajectoire qui tend vers la sortie tout en restant contrainte par la structure du labyrinthe. Les allers-retours visibles sont typiques des méthodes MCMC, qui explorent par bruit plutôt que par déterminisme.

La Figure 2 représente une carte thermique de la fréquence des positions visitées. Elle permet d'interpréter la dynamique d'exploration :

Les zones sombres indiquent des régions peu ou jamais visitées.

Les zones claires (blanc/jaune) correspondent aux positions où l'agent est resté longtemps.

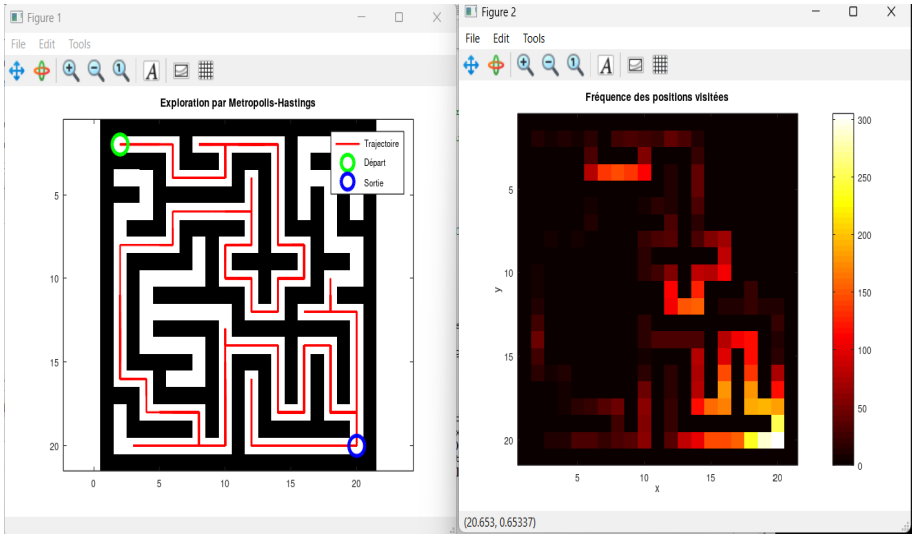


FIGURE 2.1 – exemple de résolution d'une labyrinthe

On observe une forte intensité proche de la sortie, ce qui confirme que l'algorithme converge bien vers cette région, conformément à la densité cible définie.

Cette visualisation est très utile pour évaluer la concentration de l'échantillonnage et la pertinence de l'algorithme dans un contexte d'exploration ou d'optimisation probabiliste.

On pourrait également mesurer la distribution empirique des distances à la sortie, le temps moyen d'accès, ou calculer l'entropie spatiale des positions visitées comme indicateur de dispersion. Prenons la distribution empirique des distances a la sortie qui constitue un outil essentiel pour évaluer l'efficacité d'un algorithme probabiliste d'exploration, tel que Metropolis-Hastings, dans un labyrinthe. En mesurant la fréquence à laquelle l'algorithme visite des positions situées à différentes

distances de la sortie, cette distribution permet de visualiser si l'exploration est bien orientée vers l'objectif. En effet, une concentration élevée des visites à de faibles distances indique une forte attraction vers la sortie et donc une stratégie efficace, tandis qu'une répartition plus uniforme ou des pics éloignés peuvent révéler une exploration inefficace ou désorientée. Voici l'histogramme représentant la distribution des distances à la sortie pour l'exemple traité ci-dessus. Il met en évidence que la majorité des positions visitées se situent à proximité immédiate de la sortie, ce qui témoigne de l'efficacité de la densité cible utilisée dans l'algorithme. En effet, celle-ci, construite en fonction de la distance à la sortie, guide la marche de manière cohérente vers l'objectif. Ce comportement reflète le bon fonctionnement du schéma de Metropolis-Hastings, dont la densité bien conçue favorise naturellement les zones proches de la solution.

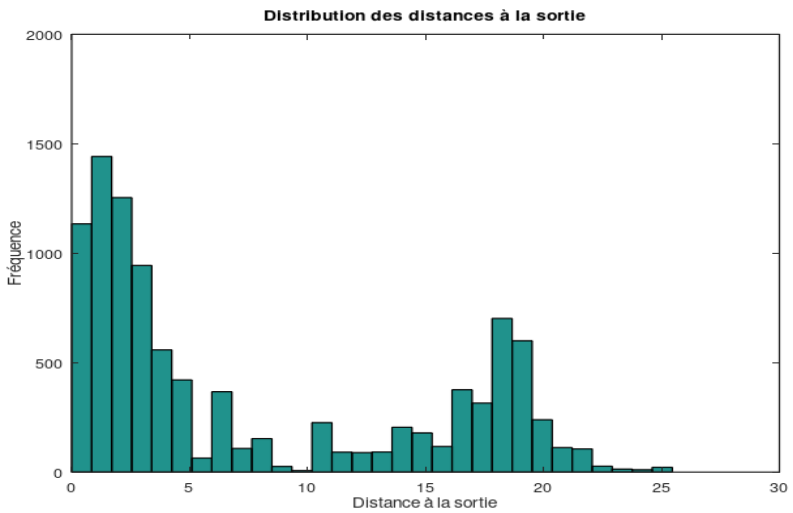


FIGURE 2.2 – l'histogramme : "Distribution des distances à la sortie"

2.4 Conclusion

Ce projet démontre l'applicabilité de l'algorithme Metropolis-Hastings à un contexte d'exploration spatiale. Bien qu'initialement conçu pour l'échantillonnage statistique, il s'adapte ici efficacement à la résolution de labyrinthes, en exploitant une densité de probabilité bien choisie. Les visualisations permettent de comprendre finement le comportement de l'agent et révèlent une convergence vers la sortie malgré la stochasticité de l'algorithme. En complément de méthodes déterministes, cette approche offre des avantages intéressants en termes de robustesse et de flexibilité dans des environnements complexes ou incertains.

Chapitre 3

Cristallographie computationnelle : Analyse Thermodynamique d'un cristal de Lennard-Jones à l'aide de Metropolis-Hastings

Introduction

La cristallographie computationnelle est un domaine de recherche interdisciplinaire qui combine la physique, la chimie, les mathématiques et l'informatique pour modéliser et prédire les propriétés des matériaux à l'échelle atomique. En s'appuyant sur des algorithmes numériques et des simulations informatiques, elle permet d'explorer la structure, la stabilité et les transformations des cristaux, en complément ou en remplacement des méthodes expérimentales classiques [5].

Dans ce projet, nous utilisons des méthodes de Monte Carlo avec l'algorithme de Metropolis pour simuler l'évolution d'un cristal en trois dimensions, modélisé par un potentiel de Lennard-Jones, qui reproduit les interactions à courte portée entre atomes neutres, comme dans le cas de l'argon solide [3]. Cette approche permet d'analyser la transition solide-liquide du système ainsi que la configuration la plus stable sous différentes conditions thermodynamiques, notamment à travers un processus de recuit simulé.

Un aspect essentiel de cette étude concerne l'apparition et l'évolution des **défauts cristallins**, tels que les lacunes ou les dislocations, qui influencent fortement les propriétés mécaniques et thermiques des matériaux. Grâce à la fonction de distribution radiale $g(r)$, nous étudions comment ces défauts émergent ou disparaissent en fonction de la température et des conditions de relaxation, et comment ils affectent la structure atomique globale du cristal.

Ce type de simulation permet non seulement de mieux comprendre les propriétés thermiques et structurales des matériaux à l'échelle microscopique, mais aussi de prédire leur comportement sans recourir à des expériences coûteuses. Ce projet s'inscrit dans une démarche de modélisation de phénomènes physiques fondamentaux, en lien avec la physique de la matière condensée et les sciences des matériaux.

3.1 Fondements théoriques

3.1.1 Potentiel de Lennard Jones : modèle d'intéractions atomiques

Le potentiel de Lennard-Jones (LJ) est l'un des modèles les plus utilisés pour représenter les interactions entre particules neutres, notamment dans les simulations de fluides simples et de gaz rares. Il combine une répulsion à courte portée et une attraction à longue portée, modélisant respectivement la répulsion de Pauli et les forces de van der Waals. Sa forme mathématique est donnée par :

$$V_{\mathrm{LJ}}(r) = 4\varepsilon \left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6} \right]$$

où :

— r est la distance entre deux particules,

— ε représente la profondeur du puits potentiel, indiquant l'intensité de l'interaction attractive,

— σ est la distance à laquelle le potentiel est nul, correspondant à la taille effective des particules.

Le terme $\left(\frac{\sigma}{r}\right)^{12}$ modélise la répulsion à courte distance due au recouvrement des nuages électroniques, tandis que $\left(\frac{\sigma}{r}\right)^{6}$ représente l'attraction à longue portée résultant des forces de dispersion.

Dans les simulations de dynamique moléculaire et de Monte Carlo, le potentiel de Lennard-Jones est largement utilisé pour étudier les propriétés thermodynamiques et structurales des systèmes simples. Il permet de modéliser des phénomènes tels que la condensation, la fusion et la formation de structures cristallines[2].

Le potentiel de Lennard-Jones est particulièrement adapté pour modéliser des systèmes où les interactions sont dominées par des forces de van der Waals, comme les gaz nobles (par exemple, l'argon). Cependant, il présente des limitations lorsqu'il s'agit de systèmes plus complexes ou de matériaux où les interactions électrostatiques, les liaisons hydrogène ou les effets directionnels jouent un rôle significatif. Dans de tels cas, des potentiels plus sophistiqués ou des modèles

multi-corps peuvent être nécessaires pour capturer fidèlement le comportement du système[2][3].

3.1.2 Unités réduites : simplification et généralisation des simulations

Dans les simulations de systèmes atomiques, l'utilisation d'unités réduites (ou unités adimensionnées) constitue une pratique standard qui permet de simplifier les équations physiques, de réduire les erreurs numériques et de rendre les résultats plus génériques. Cette approche consiste à exprimer toutes les grandeurs physiques (longueur, énergie, température, etc.) en fonction d'échelles caractéristiques du système modélisé, typiquement extraites du potentiel d'interaction.[2]

Définitions des unités réduites

Lorsqu'on utilise le potentiel de Lennard-Jones, il est courant de définir les unités réduites de la manière suivante :

— La longueur est exprimée en unités de σ, la distance à laquelle le potentiel s'annule.

— L'énergie est exprimée en unités de ε, la profondeur du puits du potentiel.

— La température est exprimée en unités de ε/k_B, où k_B est la constante de Boltzmann.

— Le temps peut être exprimé en unités naturelles dérivées, typiquement $\tau = \sigma\sqrt{m/\varepsilon}$, avec m la masse des particules.

Ainsi, en unités réduites, les équations de mouvement ou les critères d'acceptation de Monte Carlo deviennent indépendants des unités physiques réelles, ce qui facilite la comparaison entre différentes simulations.

Conversion vers les unités physiques

Le tableau suivant résume les relations de conversion entre les grandeurs physiques (en unités SI) et leurs équivalents en unités réduites :

Grandeur	**Unité réduite**	**Conversion (ex. argon)**
Longueur (L^*)	L/σ	$L^* = L$ [Å]/3.405
Énergie (E^*)	E/ε	$E^* = E$ [K $\cdot k_B$]/119.8
Température (T^*)	$k_B T/\varepsilon$	$T^* = T$ [K]/119.8
Temps (t^*)	t/τ, avec $\tau = \sigma\sqrt{m/\varepsilon}$	$\tau \approx 2.16 \times 10^{-12}$ s (argon)
Pression (P^*)	$P\sigma^3/\varepsilon$	$P^* = P$ [Pa] $\cdot\, 3.405^3/(\varepsilon/k_B)$
Vitesse (v^*)	$v/\sqrt{\varepsilon/m}$	

TABLE 3.1 – Exemples de conversions entre grandeurs physiques et unités réduites pour l'argon.

Exemple pratique

Supposons une température physique de $T = 60$ K pour de l'argon. En utilisant la valeur $\varepsilon/k_B = 119.8$ K, la température réduite devient :

$$T^* = \frac{k_B T}{\varepsilon} = \frac{60}{119.8} \approx 0.501$$

Inversement, une température réduite $T^* = 1.0$ correspond à $T = 119.8$ K en unités physiques.

Avantages et généricité

L'universalité des unités réduites permet de :

— comparer directement des résultats issus de différents systèmes Lennard-Jones (argon, krypton, etc.),

— utiliser les mêmes algorithmes sans recalibrage pour différentes conditions physiques,

— identifier plus facilement les transitions de phase en termes de seuils critiques universels (ex. : fusion de l'argon à $T^* \approx 0.7$).

3.1.3 Recuit Simulé pour la Relaxation Énergétique

Le *recuit simulé* est une méthode d'optimisation inspirée du processus physique de recuit des métaux. Dans ce processus, un matériau est chauffé à une température élevée, puis refroidi lentement pour permettre aux atomes de se réarranger et

atteindre une configuration de faible énergie. Cela permet au système d'échapper aux minima locaux et de converger vers un état plus stable[4].

Le principe physique repose sur la *dynamique thermodynamique* : à haute température, les particules ont une grande mobilité, ce qui permet au système d'explorer différents états énergétiques. À mesure que la température diminue, la mobilité des particules diminue également, favorisant la stabilisation dans une configuration à faible énergie, idéalement au minimum global.

Dans le cadre du recuit simulé, la température est généralement décrite par une loi exponentielle :

$$T(s) = T_0 \exp(-\alpha \cdot s)$$

Où :

— T_0 est la température initiale,

— α est un paramètre de déclin qui contrôle la vitesse de la diminution de la température,

— s représente l'itération courante.

Cette loi exponentielle de température permet une réduction rapide au début du processus, suivie d'un ralentissement de la diminution de la température, simulant ainsi le refroidissement progressif d'un matériau vers une configuration stable à faible énergie.

3.1.4 Fonction de Distribution Radiale

La fonction de distribution radiale, notée $g(r)$, est un outil clé en cristallographie et en physique des matériaux pour étudier la structure locale d'un système de particules. Elle mesure la probabilité relative de trouver une particule à une distance r d'une particule de référence, par rapport à une distribution aléatoire homogène. En d'autres termes, $g(r)$ permet de caractériser les interactions à courte portée entre les particules et de déterminer la structure de type cristallin ou amorphe du système.

La fonction $g(r)$ est définie comme suit :

$$g(r) = \frac{n(r)}{4\pi r^2 \rho}$$

où :

— $n(r)$ est le nombre de particules dans un intervalle de distance r à $r + dr$ autour d'une particule de référence,

— ρ est la densité de particules du système, donnée par $\rho = \frac{N}{V}$, où N est le nombre total de particules et V le volume du système,

— r est la distance radiale par rapport à la particule de référence,

— $4\pi r^2$ est le facteur géométrique pour une sphère de rayon r.

Dans le cadre des simulations de Monte Carlo et de dynamique moléculaire, la fonction $g(r)$ est souvent utilisée pour obtenir des informations sur la structure locale du système. Lors d'une simulation, on peut calculer $g(r)$ en mesurant la distance entre toutes les paires de particules et en les binant dans des intervalles de distance dr. L'histogramme ainsi obtenu permet d'estimer $g(r)$.

L'interprétation de la fonction de distribution radiale $g(r)$ permet d'obtenir des informations sur la nature du système. Par exemple :
- Si $g(r) = 1$ pour toutes les distances, le système est parfaitement aléatoire, comme dans un gaz parfait.
- Des pics successifs dans $g(r)$ à des distances $r_1, r_2, \ldots$ indiquent une structure ordonnée, comme dans un cristal. Chaque pic correspond à une couche d'atomes voisins autour de l'atome central.
- Le premier pic dans $g(r)$ est généralement lié à la distance moyenne entre les atomes voisins dans un cristal, tandis que les pics suivants reflètent les distances à des voisins plus éloignés.

En résumé, $g(r)$ fournit une description quantitative de l'arrangement des particules dans un système et est essentiel pour étudier les propriétés structurelles des matériaux.

3.2 Implémentation numérique

3.2.1 Langage et environnement utilisés

La simulation a été implémentée en `GNU Octave`, un langage de calcul scientifique similaire à `Matlab`, qui permet une manipulation efficace des tableaux et des matrices, ainsi qu'un prototypage rapide des algorithmes numériques. Ce choix est motivé par la simplicité d'écriture des scripts, les fonctions intégrées pour le traitement de données et la visualisation, et la gratuité de l'environnement.

3.2.2 Algorithme général de la simulation

Le programme réalise les étapes suivantes :

1. Génération d'une configuration initiale régulière (maille cubique ou configuration aléatoire).
2. Calcul de l'énergie initiale du système.
3. Application de l'algorithme de Monte Carlo avec recuit simulé :

— à chaque étape s, une nouvelle configuration est proposée en déplaçant un atome (ou groupe d'atomes) de façon aléatoire dans la boîte.
— la nouvelle configuration est acceptée selon le critère de Metropolis :

$$P_{\text{accept}} = \min\left(1, e^{-\Delta E/(k_B T(s))}\right)$$

— la température suit une loi décroissante de type exponentielle :

$$T(s) = T_0 \cdot \exp(-\alpha s)$$

4. Répétition du processus pendant N_{steps} itérations.
5. Sauvegarde de la configuration la plus stable (à énergie minimale).

3.2.3 Organisation du code

Le programme principal suit une structure modulaire, avec des fonctions bien séparées qui permettent d'assurer une meilleure lisibilité et maintenabilité. Les principales fonctions développées sont :

— `system_energy(positions, L)` : calcule l'énergie totale du système à partir des positions atomiques, en utilisant le potentiel de Lennard-Jones avec conditions aux limites périodiques.
— `radial_distribution(positions, L, dr, r_max)` : calcule la fonction de distribution radiale $g(r)$, qui donne des informations sur la structure locale du cristal.
— `select_group(positions, i, radius)` : sélectionne un groupe d'atomes dans un voisinage sphérique autour de l'atome i, afin de permettre un déplacement collectif lors de certaines étapes Monte Carlo.
— `lennard_jones_potential(r)` : calcule le potentiel de Lennard-Jones entre deux particules séparées par une distance r.

3.2.4 Programme principal

Le script principal de la simulation suit les étapes suivantes :

— **Initialisation des paramètres** :
  — Nombre d'itérations Monte Carlo (`N_steps`), nombre de particules (`N`), taille de la boîte (`L`), température initiale (`T0`).
  — Génération des positions initiales sur une grille régulière en 3D.
— **Visualisation de la configuration initiale** :
  — Affichage 3D des particules.

    - Calcul et traçage de la fonction de distribution radiale $g(r)$.
- **Relaxation énergétique (M essais indépendants)** :
    - Pour chaque essai :
        - Calcul de l'énergie initiale du système.
        - Application de l'algorithme de Metropolis pour déplacer une particule ou un groupe local.
        - Acceptation ou rejet des mouvements selon le critère de Metropolis.
        - Sauvegarde de l'énergie et de la configuration finale.
- **Analyse statistique des énergies finales** :
    - Moyenne et variance des énergies obtenues.
    - Calcul de l'intervalle de confiance à 95 %.
    - Sélection de la configuration la plus stable (énergie minimale).
- **Affichage des résultats finaux** :
    - Visualisation de la configuration optimale.
    - Traçage de la distribution radiale $g(r)$ correspondante.
    - Affichage du taux d'acceptation des mouvements.

3.3 Résultats et analyses

Cette section présente les résultats obtenus à partir des simulations Monte Carlo sur un système tridimensionnel d'atomes interagissant selon le potentiel de Lennard-Jones. Deux axes principaux sont abordés : l'étude de la structure cristalline stable dans un cristal contenant des défauts, et l'analyse de la transition de phase solide-liquide du système.

3.3.1 Structure la plus stable d'un cristal avec défauts

Pour étudier les effets des défauts cristallins sur la structure du matériau, une configuration initiale a été générée à partir d'un réseau cubique régulier. Un atome a ensuite été retiré aléatoirement afin d'introduire un défaut ponctuel. Le système a été relaxé via l'algorithme de Metropolis à température décroissante en utilisant la méthode de recuit simulé, afin d'atteindre une configuration d'énergie minimale.

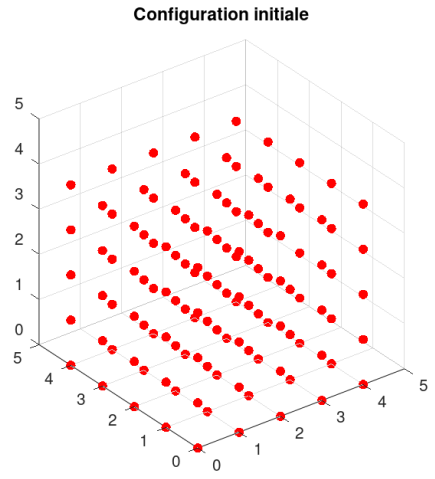


FIGURE 3.1 – Configuration initiale avec défaut cristallin.

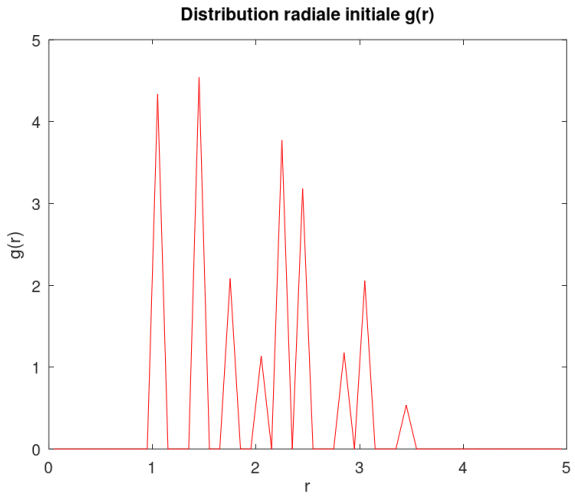


FIGURE 3.2 – configuration initiale avec défaut cristallin.

La distribution radiale $g(r)$ associée à la configuration finale permet d'identifier la structuration locale du cristal et de vérifier la persistance d'un ordre à courte portée malgré le défaut.

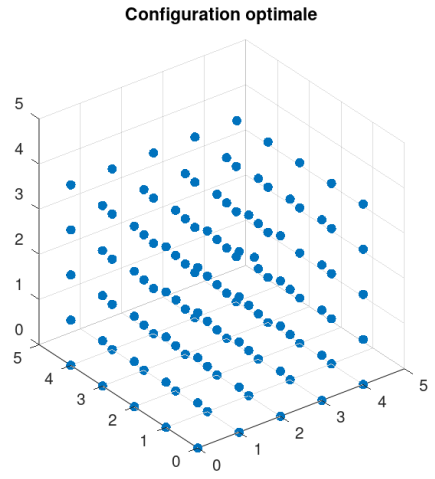


FIGURE 3.3 – Structure la plus stable obtenue après relaxation.

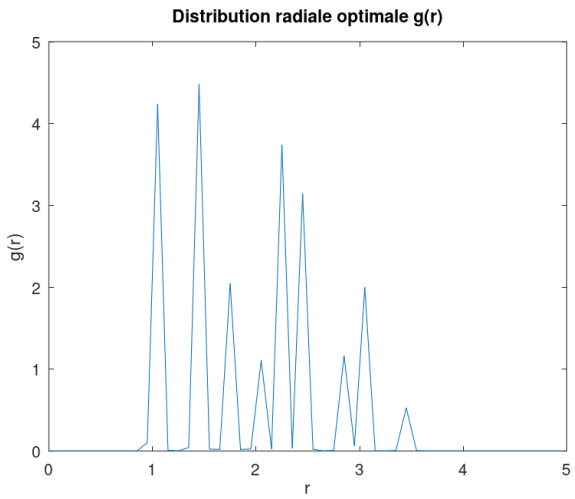


FIGURE 3.4 – Distribution radiale $g(r)$ de la configuration optimisée.

Interprétation des résultats. Après la relaxation du cristal initialement introduit avec un défaut (Figure (3.1)), la configuration finale (Figure (3.3)) montre une structure globalement ordonnée, mais avec des irrégularités localisées. Ces perturbations dans le réseau atomique sont la conséquence directe du défaut initial (par exemple, un atome manquant ou déplacé), et leur persistance après relaxation indique que le système a trouvé un minimum local d'énergie dans un état désordonné partiel.

La fonction de distribution radiale finale $g(r)$ (Figure (3.4)) présente des pics

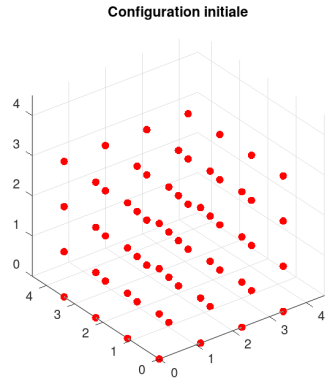
nets, similaires à ceux d'un cristal parfait, mais avec une atténuation légère et un élargissement de certains pics. Cela traduit un ordre à courte et moyenne portée maintenu, tout en reflétant la perturbation introduite par le défaut. L'absence d'un retour à une symétrie parfaite démontre que le défaut est stabilisé dans la structure optimisée.

Ces résultats montrent que le système conserve sa nature solide tout en intégrant le défaut dans une configuration d'énergie minimale, révélant ainsi la capacité du réseau cristallin à accommoder des irrégularités structurelles.

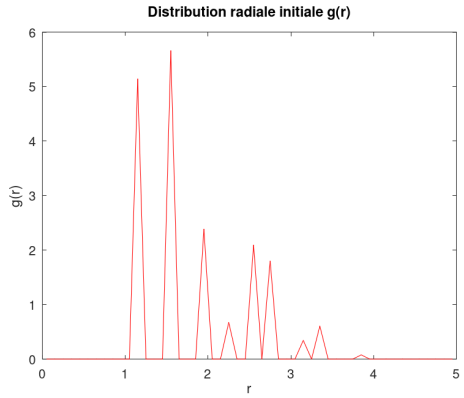
3.3.2 Transition de phase solide-liquide

L'objectif de cette section est d'observer la transition de phase d'un cristal (comme l'argon solide) tridimensionnel à travers une simulation Monte Carlo utilisant l'algorithme de Metropolis, sans recours au recuit simulé. Le système évolue à température constante, et nous étudions la structure avant et après l'évolution pour identifier les changements liés à la fusion ou à la désorganisation du réseau cristallin.

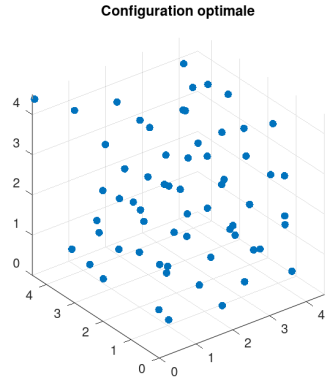
Les figures ci-dessous présentent la configuration initiale du cristal, la fonction de distribution radiale $g(r)$ correspondante, ainsi que la configuration finale après un certain nombre d'itérations et sa distribution radiale.



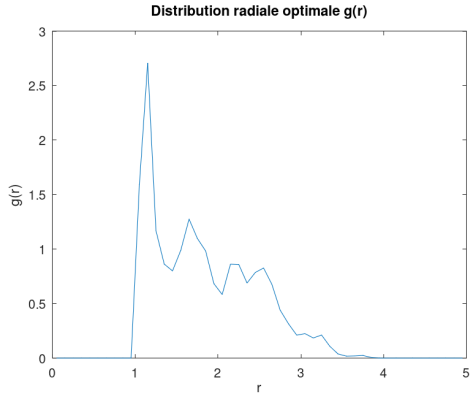
(a) Configuration initiale



(b) Distribution radiale initiale $g(r)$



(c) Configuration finale après fusio



(d) Distribution radiale finale $g(r)$

FIGURE 3.5 – Évolution de la structure cristalline et de la fonction $g(r)$ au cours de la simulation.

Interprétation des résultats. La configuration finale illustrée en Figure (c) montre une désorganisation notable par rapport à la configuration initiale ordonnée. Les atomes ne suivent plus une disposition régulière, ce qui est caractéristique d'une phase liquide. Ce désordre spatial est une signature visuelle de la transition de phase solide-liquide.

La fonction de distribution radiale finale $g(r)$ (Figure (d)) confirme cette transition. Alors que dans la phase solide $g(r)$ présente des pics nets et bien définis, caractéristiques d'un ordre à longue portée, la courbe finale devient plus lissée avec un pic principal élargi suivi de faibles oscillations. Cela indique une perte d'ordre à longue distance et une organisation locale typique d'un liquide.

Ces résultats montrent que le système initialement cristallin évolue vers une

phase désordonnée sous l'effet des fluctuations thermiques, simulant ainsi une fusion du cristal.

3.4 Conclusion

Dans ce projet, nous avons mis en œuvre une simulation de Monte Carlo basée sur l'algorithme de Metropolis pour étudier deux phénomènes fondamentaux de la physique des solides : l'effet des défauts cristallins sur la stabilité structurale et la transition de phase solide-liquide. Le système modélisé repose sur un ensemble d'atomes en interaction selon le potentiel de Lennard-Jones, permettant de reproduire les caractéristiques essentielles des solides atomiques simples comme l'argon.

Dans la première partie, l'introduction d'un défaut ponctuel dans la structure cristalline nous a permis d'observer la relaxation du système vers une configuration d'énergie minimale. Malgré la perturbation initiale, la structure finale conserve un ordre local, comme le montre la fonction de distribution radiale $g(r)$, tout en présentant des modifications spécifiques induites par l'irrégularité.

La seconde partie de l'étude a permis d'analyser la transition de phase solide-liquide d'un cristal initialement ordonné soumis à une élévation de température. L'analyse visuelle des configurations et l'évolution de la fonction $g(r)$ confirment la perte progressive de l'ordre à longue portée caractéristique de la fusion, marquant le passage à une phase liquide désordonnée.

Ces résultats démontrent l'efficacité des approches numériques pour explorer des phénomènes structuraux et thermodynamiques à l'échelle atomique. Ils constituent une base solide pour des investigations plus poussées, incluant d'autres types de défauts, des potentiels plus complexes, ou encore des conditions expérimentales plus proches du réel.

Chapitre 4

Prédiction bayésienne du prix d'une cryptomonnaie par échantillonnage MCMC : Application de l'algorithme de Metropolis-Hastings

4.1 Introduction

Les marchés financiers modernes, et en particulier ceux des cryptomonnaies, sont caractérisés par une volatilité extrême, une dynamique difficilement prévisible et une grande incertitude. Dans ce contexte, les méthodes traditionnelles de prévision, fondées sur des moyennes ou des modèles déterministes, montrent rapidement leurs limites. Il devient donc crucial de s'appuyer sur des approches probabilistes permettant de capturer l'incertitude inhérente aux prévisions. L'algorithme de Metropolis-Hastings, appartenant à la famille des méthodes de Monte Carlo par chaînes de Markov (MCMC), permet d'échantillonner efficacement des distributions a posteriori complexes, même lorsque leur forme analytique est difficilement exploitable. Cette extension s'interesse à la modélisation bayésienne du prix futur du Bitcoin, une des cryptomonnaies les plus suivies. Plutôt que de fournir une estimation unique, nous cherchons à obtenir une distribution prédictive du prix, à partir des rendements observés sur une période donnée. Pour ce faire, nous supposons que les rendements journaliers suivent une loi normale, dont les paramètres sont inconnus et estimés par échantillonnage MCMC via l'algorithme de Metropolis-Hastings.
Cette approche permet d'intégrer l'incertitude sur les paramètres dans la prédiction

du prix futur, et fournit non seulement une estimation centrale, mais aussi un intervalle de confiance crédible. Elle offre ainsi une alternative robuste aux approches naïves, tout en illustrant la puissance des méthodes bayésiennes dans des contextes financiers complexes.

4.2 Fondements théoriques

4.2.1 Loi normale pour les rendements

Dans notre modèle, nous supposons que les rendements journaliers r_t suivent une distribution normale :

$$r_t \sim \mathcal{N}(\mu, \sigma^2)$$

où μ représente le rendement moyen et σ^2 la variance (ou volatilité) des rendements. Cette hypothèse est couramment utilisée dans les modèles financiers à court terme, notamment pour les actifs volatils comme le Bitcoin. Bien que les rendements de cryptomonnaies puissent présenter des queues épaisses ou des asymétries, la loi normale fournit un cadre de départ simple et interprétable.

4.2.2 Vraisemblance et fonction d'énergie

La vraisemblance des observations $(r_1, r_2, \ldots, r_n)$ conditionnellement aux paramètres μ et σ est donnée par[6] :

$$L(\mu, \sigma) = \prod_{t=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(r_t - \mu)^2}{2\sigma^2}\right)$$

En prenant le logarithme, on obtient la log-vraisemblance :

$$\log L(\mu, \sigma) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{n}(r_t - \mu)^2$$

Dans le cadre du Metropolis-Hastings, on définit une fonction d'énergie associée comme l'opposée de la log-vraisemblance :

$$E(\mu, \sigma) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{t=1}^{n}(r_t - \mu)^2$$

Cette fonction d'énergie permet de comparer la qualité des propositions de paramètres μ et σ. Plus l'énergie est faible, plus la proposition est cohérente avec les données observées.

4.2.3 Algorithme de Metropolis-Hastings

L'algorithme de Metropolis-Hastings permet d'échantillonner des paramètres (μ, σ) à partir de la distribution a posteriori induite par les données et la loi de vraisemblance. Le principe est le suivant :

1. À partir d'un état courant (μ, σ), on propose un nouvel état (μ', σ') en ajoutant un bruit aléatoire selon une loi symétrique.
2. On calcule la différence d'énergie entre les deux états, puis le taux d'acceptation :
$$\alpha = \min\left(1, \exp\left(\frac{E_{\text{actuel}} - E_{\text{proposé}}}{T}\right)\right)$$
3. On accepte la proposition avec probabilité α ; sinon, on conserve l'état actuel.

Le paramètre $T > 0$ est une température, inspirée du recuit simulé, qui contrôle l'exploration de l'espace des paramètres. Plus T est élevé, plus la fluctuation est importante et plus l'algorithme accepte des propositions éloignées, ce qui favorise une exploration large. À l'inverse, une température plus basse conduit à une fluctuation faible donc une acceptation plus stricte, ce qui peut être utile pour raffiner les estimations.

4.3 Mise en œuvre algorithmique

4.3.1 Génération des données

Pour les besoins de cette simulation, nous générons une série de rendements journaliers r_t sur 180 jours. Ces rendements sont simulés selon une loi normale centrée autour d'une moyenne $\mu = 0.002$ et d'un écart-type $\sigma = 0.02$, représentant une volatilité modérée :

$$r_t \sim \mathcal{N}(0.002, 0.02^2), \quad t = 1, \ldots, 180$$

Le dernier prix connu du Bitcoin est supposé être $P_0 = 95\,000$ USD, et les rendements serviront à prédire le prix P_{suivant} :

$$P_{\text{t+1}} = P_{\text{t}} \cdot (1 + r_{\text{t+1}}) \tag{4.3.1}$$

P_{t} : Prix du Bitcoin au jour t.
$P_{\text{t+1}}$: Prix du Bitcoin au jour $t + 1$.
$r_{\text{t+1}}$: Rendement du Bitcoin au jour $t + 1$.

4.3.2 Paramètres et initialisation

Les paramètres utilisés dans la simulation sont les suivants :

— Nombre de jours simulés : $n = 180$

— Espérance réelle des rendements : $\mu_{\text{réel}} = 0{,}002$

— Écart-type réel des rendements : $\sigma_{\text{réel}} = 0{,}02$

— Vecteur des rendements passés : $r = (r_1, r_2, \ldots, r_{180})$

— Prix actuel du Bitcoin : $P_{\text{dernier}} = 95\,000$ USD

— Nombre total d'itérations : $N = 50\,000$

— Burn-in (nombre d'échantillons ignorés en début de chaîne) : 5 000

— Température initiale : $T = 0{,}05$

— Température minimale : $T_{\min} = 0{,}001$

— Seuil d'énergie : $E_{\text{seuil}} = -447$

— Valeur initiale de l'espérance dans l'échantillon : $\mu = 0$

— Valeur initiale de la volatilité dans l'échantillon : $\sigma = 0{,}02$

— Pas de perturbation pour la proposition : $\epsilon = 0{,}05$

Ces paramètres ont été choisis de manière à garantir un bon équilibre entre l'exploration de l'espace des paramètres et la convergence rapide de la chaîne de Markov vers une distribution stationnaire. L'algorithme utilise une loi de proposition gaussienne symétrique pour perturber les paramètres :

$$\mu' = \mu + \epsilon \cdot \mathcal{N}(0,1), \quad \sigma' = |\sigma + \epsilon \cdot \mathcal{N}(0,1)|$$

4.3.3 Implémentation du Metropolis-Hastings

L'algorithme de Metropolis-Hastings a été implémenté en Octave, un langage particulièrement adapté pour les calculs matriciels et les simulations statistiques. À chaque itération de l'algorithme, les étapes suivantes sont effectuées :

1. Proposition d'un nouveau couple de paramètres (μ', σ') à partir de l'état actuel de la chaîne.
2. Calcul de l'énergie associée à cette proposition.
3. Acceptation ou rejet de la proposition selon la règle de Metropolis-Hastings, en fonction de l'énergie et de la température T.
4. Enregistrement des échantillons acceptés pour analyse postérieure.

```
function [energies,samples]=Metropolis(mu,sigma,epsilon,T,r,E_seuil,T_min,N)
    samples=zeros(N,2);
    energies=zeros(N,1);
    for i=1:N
      mu_p=mu+randn*epsilon;
      sigma_p=abs(sigma+randn*epsilon);

      E_current=EnergyC(r,mu,sigma);
      E_proposed=EnergyC(r,mu_p,sigma_p);
      alpha=min(1,exp((E_current-E_proposed)/T));
      if rand<alpha
        mu=mu_p;
        sigma=sigma_p;
      end
      energies(i)=EnergyC(r,mu,sigma);
      samples(i,:)=[mu,sigma];

      if energies(i)<E_seuil && T>T_min
        T=T*0.9;
      end
    end
endfunction
```

Listing 4.1 – Script `Metropolis-Hastings`

4.3.4 Stockage des échantillons et prédiction

À chaque itération, les paires (μ, σ) acceptées sont stockées dans une matrice `samples` et les énérgies dans un vecteur `energies` . Une fois la phase de burn-in éliminée, on tire des rendements futurs $r_{\text{suivant}} \sim \mathcal{N}(\mu, \sigma^2)$ pour chaque couple échantillonné, et on en déduit les prix prévus P_{suivant} via (4.3.1). Ces prédictions permettent d'établir un intervalle de confiance sur le prix futur du Bitcoin grâce à l'algorithme de Monte-Carlo et d'estimer la probabilité de dépassement d'un seuil qui vaut 100 000 USD dans cette simulation.

4.3.5 Visualisations

Pour évaluer la dynamique de convergence et les distributions obtenues, plusieurs graphiques sont produits à partir des échantillons stockés :

— Évolution de μ et σ au fil des itérations

— Fonction d'énergie à chaque itération

— Histogrammes des valeurs postérieures de μ et σ

— Histogramme du prix P_{suivant} calculé à partir des échantillons

Ces visualisations permettent de juger de la stabilisation de la chaîne et d'estimer les quantiles de la distribution prédictive.

4.4 Analyse des résultats

4.4.1 Convergence de la chaîne

L'analyse des courbes de trajectoire pour les paramètres μ, σ et l'énergie comme on le voit dans les figures 4.1 et 4.2 permet d'évaluer la stabilité de la chaîne de Markov. Après une phase transitoire (burn-in), les valeurs semblent osciller autour d'un régime stationnaire, signe d'une bonne convergence.

— **Trace de** μ : les fluctuations se stabilisent après les premières itérations, autour de la moyenne estimée.

— **Trace de** σ : la volatilité converge également vers une zone restreinte.

— **Énergie** : la décroissance initiale de l'énergie puis sa stabilisation confirme que l'algorithme atteint des configurations à haute vraisemblance.

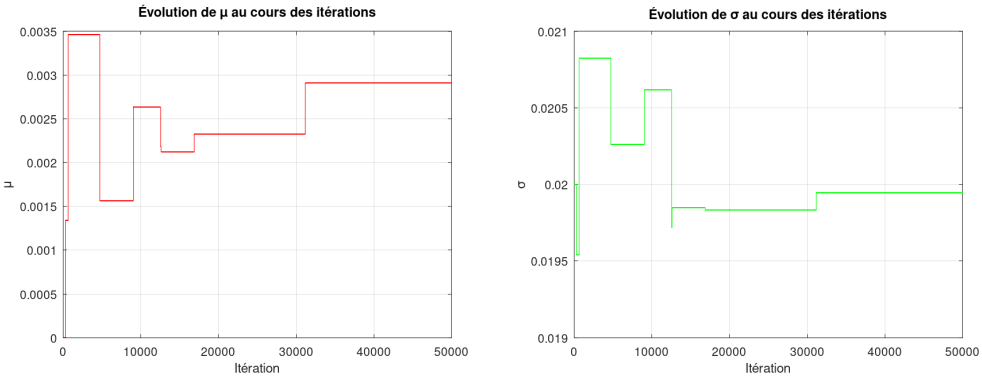


FIGURE 4.1 – Évolution des paramètres μ et σ au cours des itérations

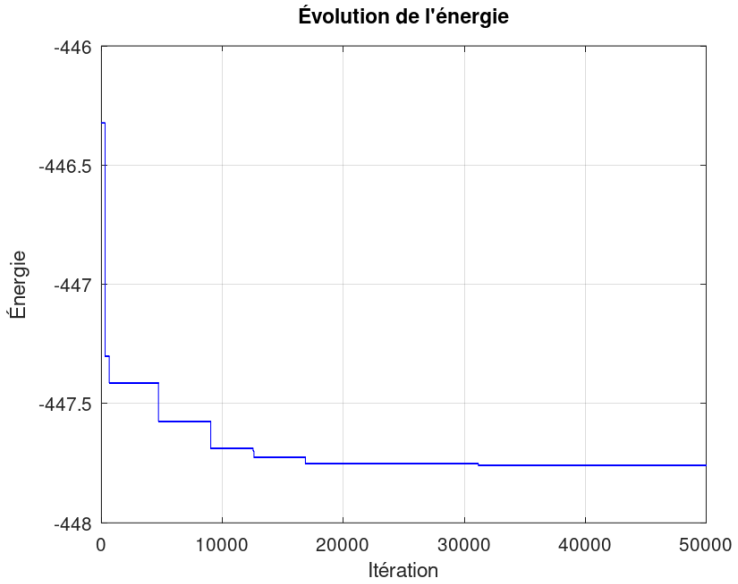


FIGURE 4.2 – Évolution de l'énergie au cours des itérations

4.4.2 Résultats statistiques

Après suppression des 5 000 premières itérations (burn-in), les échantillons conservés permettent d'estimer les caractéristiques statistiques de la distribution a posteriori.
En utilisant l'alogorithme de Monte-Carlo, on déduit les valeurs de la moyenne et l'écart-type du tableau 4.1 sont proches des paramètres réels utilisés pour générer les rendements ($\mu = 0.002$, $\sigma = 0.02$), ce qui valide la qualité de l'estimation. Ces résultats montrent également la capacité du modèle bayésien à fournir une estimation probabiliste du prix, en intégrant l'incertitude sur les paramètres.

	Estimation	**Intervalle de confiance à 95 %**
Moyenne	$\hat{\mu} = 0.00273$	[0.00272 ; 0.00273]
Ecart-type	$\hat{\sigma} = 0.02008$	[0.02008 ; 0.02009]
Prix suivant	P_{suivant}=95267.41	[95 249.83 ; 95 284.99]

TABLE 4.1 – Estimations et IC à 95 % des paramètres statistiques de la distribution.

4.5 Comparaison avec un modèle naïf

Pour mettre en perspective les apports du modèle bayésien fondé sur Metropolis-Hastings, nous le comparons à une approche plus simple : la prédiction naïve.
Le modèle naïf consiste à estimer le rendement futur μ par la moyenne empirique des rendements observés, et à supposer que cette valeur représente fidèlement le comportement futur du marché. La prédiction du prix est alors donnée par :

$$P_{\text{naïf}} = P_0 \cdot (1 + \bar{r}) \ = 95\ 243.04\ USD$$

où $\bar{r}$ est la moyenne des 180 rendements simulés.

4.5.1 Comparaison des résultats

- **Prix prédit** : le modèle naïf fournit une valeur unique, tandis que l'approche bayésienne fournit une distribution complète du prix futur comme l'indique la figure ci-dessous.
- **Incertitude** : le modèle naïf ignore totalement la variabilité autour de μ, contrairement au modèle bayésien qui intègre cette incertitude via l'échantillonnage de (μ, σ).
- **Robustesse** : en situation de forte volatilité, le modèle bayésien est plus adapté, car il ne repose pas sur une estimation ponctuelle mais sur un ensemble d'hypothèses pondérées.

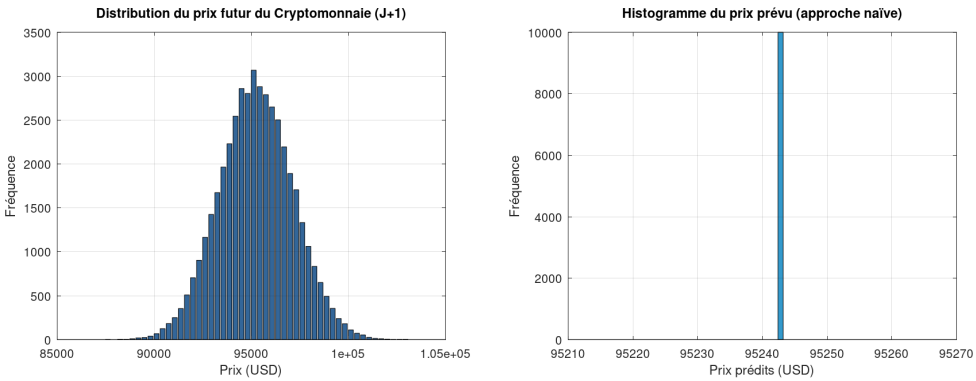


FIGURE 4.3 – Comparaison entre la distribution du prix futur de Bitcoin au jour suivant pour le modele naïf et l'approche bayésienne.

Cette comparaison 4.3 met en évidence la valeur ajoutée d'un cadre probabiliste, notamment lorsqu'il s'agit de prendre en compte le risque et l'incertitude dans la prévision.

4.6 Discussion et limites

Ce projet met en évidence l'efficacité de l'algorithme de Metropolis-Hastings pour estimer une distribution a posteriori des paramètres (μ, σ) et produire des prédictions probabilistes du prix futur du Bitcoin. Cependant, plusieurs limitations doivent être prises en compte.

4.6.1 Sensibilité aux hyperparamètres

Les performances de l'algorithme dépendent fortement du choix des paramètres :

— Le nombre d'itérations N influence la convergence de la chaîne. Un nombre insuffisant peut mener à des résultats peu fiables.

— La température T joue un rôle central dans l'acceptation des propositions. Si elle est trop élevée, les mauvaises propositions sont trop souvent acceptées ; trop basse, la chaîne stagne.

— Le pas ϵ conditionne la dispersion des propositions. Un mauvais réglage peut ralentir l'exploration ou engendrer une faible acceptation.

4.6.2 Simplifications du modèle

Le modèle repose sur l'hypothèse que les rendements sont indépendants, identiquement distribués, et suivent une loi normale. Bien que courante en finance quantitative, cette hypothèse est simplificatrice. En pratique, les rendements des actifs financiers présentent souvent des caractéristiques plus complexes, telles qu'une asymétrie, une leptokurticité (queues épaisses) ou encore une autocorrélation temporelle. De plus, le modèle suppose implicitement que les propriétés statistiques des rendements, notamment leur espérance et leur variance, demeurent stationnaires sur toute la période considérée (ici 180 jours). Or, cette hypothèse de stationnarité peut s'avérer irréaliste dans un marché volatil et soumis à des régimes changeants comme celui des cryptomonnaies.

4.6.3 Axes d'amélioration

Plusieurs pistes peuvent être envisagées pour enrichir le modèle. Tout d'abord, l'adoption d'un modèle hétéroscédastique tel que GARCH permettrait de mieux capturer la volatilité temporelle des rendements, ce qui constitue une amélioration significative par rapport à l'hypothèse de variance constante[1]. Ensuite, l'intégration de données réelles, notamment des séries historiques de prix du Bitcoin, renforcerait la validité empirique du modèle et permettrait une évaluation plus réaliste des performances prédictives.

Par ailleurs, une extension naturelle du modèle serait de le généraliser à des prévisions sur plusieurs jours, ou encore d'y inclure des variables explicatives exogènes comme des indicateurs macroéconomiques ou des signaux techniques issus du marché. Enfin, une exploration d'autres lois de proposition, telles que la loi de Cauchy, ou l'utilisation de techniques adaptatives permettrait d'améliorer l'efficacité de l'échantillonnage et la convergence de la chaîne de Markov.

4.7 Conclusion

Dans cette extension, nous avons appliqué l'algorithme de Metropolis-Hastings à un problème concret de prévision : estimer le prix futur du Bitcoin à partir d'un modèle bayésien basé sur les rendements journaliers. En modélisant ces rendements comme une loi normale de paramètres inconnus, nous avons pu construire une chaîne de Markov dont les états successifs reflètent des échantillons a posteriori de (μ, σ). Cette approche probabiliste nous a permis de générer une distribution prédictive du prix futur, accompagnée d'intervalles de confiance et d'une estimation de la probabilité de dépassement d'un seuil donné. L'algorithme s'est montré efficace pour explorer l'espace des paramètres, à condition de calibrer soigneusement les hyperparamètres tels que la température et le pas de proposition. Les résultats obtenus mettent en lumière l'intérêt des méthodes MCMC dans les situations d'incertitude élevée, comme les marchés cryptographiques. Bien que simplifié, le modèle illustre la puissance de l'inférence bayésienne pour quantifier l'incertitude dans les prévisions. Enfin, ce travail ouvre la voie à de nombreuses extensions : modèles de volatilité plus réalistes, données réelles, inférence séquentielle, ou application à d'autres actifs financiers. L'algorithme de Metropolis-Hastings s'affirme ainsi comme un outil polyvalent dans la modélisation probabiliste moderne.

Conclusion Générale

À travers ce projet, nous avons exploré la richesse et la polyvalence de l'algorithme de Metropolis-Hastings, en l'appliquant à quatre contextes très différents mais unis par une même exigence : la gestion de l'incertitude dans des espaces de solution vastes et complexes. Chaque chapitre a mis en lumière un aspect particulier de cette méthode d'échantillonnage, confirmant son efficacité pour résoudre des problèmes concrets là où les approches classiques trouvent leurs limites.
Dans le premier chapitre, la résolution probabiliste de grilles de Sudoku a illustré comment Metropolis-Hastings peut guider la recherche de solutions valides dans un espace combinatoire fortement contraint.
Le deuxième chapitre, a démontré l'adaptabilité de l'algorithme à des problèmes de navigation spatiale. En utilisant une densité cible dépendant de la distance à la sortie, l'algorithme a permis de simuler une exploration stochastique efficace, révélant des trajectoires plausibles malgré la complexité du labyrinthe. Cette application souligne la robustesse de la méthode dans des environnements contraints et incertains.
Le troisième chapitre a transporté l'algorithme dans le domaine de la physique statistique, en simulant l'évolution énergétique d'un cristal de Lennard-Jones. Cette modélisation a permis d'observer à la fois la relaxation d'un réseau cristallin avec défauts, et une transition de phase solide-liquide, confirmée par l'analyse de la distribution radiale $g(r)$.
Enfin, le dernier chapitre a abordé un domaine à fort enjeu contemporain : la prévision des prix des cryptomonnaies. En modélisant les rendements journaliers comme des variables aléatoires, Metropolis-Hastings a permis d'estimer une distribution a posteriori des paramètres, conduisant à des prévisions probabilistes du prix futur du Bitcoin. Cette approche met en valeur la puissance de l'inférence bayésienne dans des environnements financiers incertains.

Au-delà des résultats spécifiques, ce projet souligne la valeur ajoutée des méthodes de Monte Carlo et de l'inférence probabiliste pour modéliser, simuler et analyser des phénomènes complexes. L'algorithme de Metropolis-Hastings apparaît ainsi comme un outil transversal, adapté à de nombreux domaines allant de la combinatoire à la physique, en passant par l'intelligence artificielle et la finance.

Parmi les perspectives ouvertes par ce travail, on peut citer l'optimisation des lois de proposition, l'intégration de données réelles, l'extension à des modèles dynamiques (chaînes temporelles, modèles séquentiels), ou encore l'application de méthodes MCMC plus avancées (algorithmes adaptatifs, Hamiltonian Monte Carlo). Ce projet constitue ainsi une base solide pour aborder des problématiques plus complexes, en alliant rigueur mathématique, modélisation réaliste et capacité de simulation.

Bibliographie

[1] Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3) :307–327, 1986.

[2] Daan Frenkel and Berend Smit. *Understanding Molecular Simulation : From Algorithms to Applications*. Computational Science. Academic Press, San Diego, California, 2nd edition, 2002.

[3] Jean-Pierre Hansen and I.R. McDonald. *Theory of Simple Liquids*. Academic Press, Amsterdam, 3rd edition, 2006.

[4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598) :671–680, 1983.

[5] Ivan A. Kruglov, Alexey V. Yanilkin, Yana Propad, Arslan B. Mazitov, Pavel Rachitskii, and Artem R. Oganov. Crystal structure prediction at finite temperatures. *npj Computational Materials*, 9 :197, 2023.

[6] Marco Taboga. Normal distribution – maximum likelihood estimation, 2021. Lectures on Probability Theory and Mathematical Statistics, Kindle Direct Publishing. Online appendix.